

Programmation annuelle de Mathématiques – Terminale STI2D (spécialité PCM) Année 2026–2027 – Zone C (3 h/semaine)

Version du 27 juin 2026

Présentation générale

Ce document propose une organisation spiralaire de l’enseignement de la partie mathématiques de la spécialité « Physique-Chimie et Mathématiques » en terminale STI2D, conforme au programme officiel. L’année comporte 31 semaines de cours effectifs (hors vacances et jours fériés). Chaque notion est introduite une première fois, puis réinvestie et approfondie dans un second temps, en lien avec les autres thèmes et avec la physique-chimie. Cette approche favorise une acquisition progressive et durable des connaissances.

Place de Python : La programmation est intégrée de façon transverse et évaluée dans *tous* les sujets (formatives, contrôles, DM, PBL). Les compétences algorithmiques (boucles, tests, fonctions, simulation, calcul approché, méthodes numériques) sont développées en parallèle des mathématiques et réactivées régulièrement selon le même principe spiralé.

Utilisation de l’IA dans les DM : Dans chaque devoir maison, l’élève est invité à utiliser un outil d’intelligence artificielle générative (ChatGPT, Mistral, Copilot, etc.) pour :

- obtenir des explications ou des exemples supplémentaires sur une notion,
- suggérer des pistes de résolution,
- déboguer ou améliorer un code Python,
- confronter différentes méthodes de résolution.

L’IA ne doit cependant pas rédiger la solution finale à la place de l’élève. Le raisonnement personnel et la rédaction restent exigibles. Un **journal de bord** (questions posées, réponses obtenues, critiques et adaptations apportées) est à annexer à chaque DM. Cette pratique développe l’esprit critique et la capacité à évaluer la pertinence des réponses générées.

1 Tableau récapitulatif des contenus à enseigner

Le tableau ci-dessous reprend l'intégralité des attendus du programme de mathématiques. La colonne « **Traité** » indique si le contenu a été abordé en classe (coche ✓), et la colonne « **Reprise** » signale les notions qui feront l'objet d'un second passage (◐) pour consolider la compréhension.

Thème / Contenu	Traité	Reprise
Analyse – Intégration		
Définition de l'intégrale (aire sous la courbe), notation $\int_a^b f(x)dx$	☐	☐
Approximation par la méthode des rectangles, lien somme $\leftrightarrow$ intégrale	☐	☐
Intégrale d'une fonction négative, extension aux fonctions de signe quelconque	☐	☐
Propriétés : linéarité, positivité, croissance, relation de Chasles	☐	☐
Valeur moyenne d'une fonction sur $[a; b]$	☐	☐
Fonction $F(x) = \int_a^x f(t)dt$: dérivée et primitive	☐	☐
$\int_a^b f(x)dx = F(b) - F(a)$ où F est une primitive de f	☐	☐
Fonction exponentielle de base e		
Nombre e , fonction $x \mapsto e^x$, dérivée, courbe, limites	☐	☐
Dérivée de $x \mapsto e^{kx}$, croissance comparée ($x^n e^{-x}$)	☐	☐
Fonction logarithme népérien		
Définition : $\ln a$ solution de $e^x = a$, sens de variation, courbe, limites	☐	☐
Propriétés algébriques : $\ln(ab)$, $\ln(a/b)$, $\ln(a^n)$, $\ln(\sqrt{a})$, $\ln(a^x)$	☐	☐
Lien avec $\log_{10}$	☐	☐
Résolution d'équations/inéquations avec exponentielle et logarithme	☐	☐
Équations différentielles		
Notion d'équation différentielle, solution	☐	☐
Équations $y' = ay$ et $y' = ay + b$: ensembles de solutions, condition initiale	☐	☐
Composition de fonctions		
Définition $v \circ u$, dérivée : $(v \circ u)' = u' \times (v' \circ u)$	☐	☐
Dérivée de $x \mapsto (u(x))^n$, $\cos(u(x))$, $\sin(u(x))$, $e^{u(x)}$, $\ln(u(x))$	☐	☐
Primitives de $u' \cdot u^n$, $u' \cdot e^u$, $u' \cdot \cos u$, $u' \cdot \sin u$	☐	☐
Nombres complexes		
Exponentielle complexe : $e^{i\theta} = \cos \theta + i \sin \theta$	☐	☐
Forme exponentielle $z = re^{i\theta}$ ($r > 0$), passage algébrique $\leftrightarrow$ exponentielle	☐	☐
Formules d'addition et de duplication, linéarisation de $\cos^2 \theta$ et $\sin^2 \theta$	☐	☐
Expression complexe des translations, rotations, homothéties	☐	☐
Résolution d'équation du premier degré, de $z^2 = a$ ($a \in \mathbb{R}$)	☐	☐

2 Calendrier et programmation harmonieuse

2.1 Calendrier retenu (zone C, jours ouvrés)

Les vacances et jours fériés ont été pris en compte. Les semaines de cours sont les suivantes :

Période	Dates	Nb de semaines
Rentrée – Toussaint	1er sept. – 16 oct. 2026	7
Toussaint – Noël	2 nov. – 18 déc. 2026	7
Noël – Hiver	4 janv. – 5 fév. 2027	5
Hiver – Printemps	22 fév. – 2 avr. 2027	6
Printemps – Fin mai	19 avr. – 31 mai 2027	6
Total		31 semaines

Jours fériés (tombant un jour de cours) exclus : 1er janvier 2027 (vendredi), 29 mars 2027 (lundi de Pâques), 6 mai 2027 (Ascension, jeudi), 17 mai 2027 (Pentecôte, lundi). Les 1er et 8 mai 2027 tombent un samedi.

2.2 Programmation par trimestre (approche spiralée)

Les tableaux suivants précisent pour chaque semaine :

- les contenus nouveaux,
- les **points ciblés dans l'évaluation formative** (5 à 10 min en début de séance) incluant systématiquement une question Python,
- des remarques utiles.

Les évaluations formatives mêlent mathématiques et programmation, avec réactivation des notions antérieures.

2.2.1 1^{er} trimestre (septembre – novembre 2026) – 14 semaines

Sem.	Contenus nouveaux	Évaluation formative – points évalués (dont Python)	Remarques
1-2	Intégration : définition, aire sous la courbe, méthode des rectangles.	Calcul d'aires simples (rectangle, trapèze). Python : programme pour la méthode des rectangles (somme de Riemann).	Lien avec l'énergie (physique).
3-4	Propriétés de l'intégrale : linéarité, positivité, croissance, relation de Chasles. Valeur moyenne.	Spirale : méthode des rectangles (S1-2). Python : calcul de la valeur moyenne par simulation.	Lien avec la puissance moyenne en électricité.
5-6	Fonction $F(x) = \int_a^x f(t)dt$: dérivée, primitive. Théorème fondamental.	Spirale : propriétés de l'intégrale (S3-4). Python : vérification numérique de la dérivée de F .	Lien avec la cinématique (position, vitesse).
7-8	Calcul d'intégrales à l'aide de primitives. Primitives des fonctions usuelles (polynômes, $1/x$, cos, sin).	Spirale : dérivation (1ère). Python : fonction qui calcule une primitive d'un polynôme.	
9-10	Fonction exponentielle : introduction du nombre e , fonction e^x , dérivée, courbe, limites.	Spirale : fonctions a^x (1ère). Python : recherche de e par balayage/dichotomie.	Lien avec la radioactivité (décroissance).
11-12	Dérivée de e^{kx} , croissance comparée ($x^n e^{-x}$). Étude de fonctions.	Spirale : exponentielle (S9-10). Python : tracé de courbes avec exponentielle.	
13-14	Logarithme népérien : définition, sens de variation, courbe, limites.	Bilan T1 : intégration, exponentielle, logarithme. Python : calcul de ln avec math.	Lien avec le pH, les décibels.

Contrôles continus : semaines 5, 10, 14 (avec exercice Python). **Formatives :** chaque semaine.

2.2.2 2^e trimestre (novembre 2026 – février 2027) – 12 semaines

Sem.	Contenus nouveaux	Évaluation formative – points évalués (dont Python)	Remarques
15-16	Propriétés algébriques du logarithme népérien : $\ln(ab)$, $\ln(a/b)$, $\ln(a^n)$, $\ln(\sqrt{a})$, $\ln(a^x)$.	Spirale : logarithme (S13-14). Python : vérification numérique des propriétés.	Lien avec le logarithme décimal.
17-18	Résolution d'équations/inéquations avec e^x et $\ln x$. Étude de fonctions composées.	Spirale : exponentielle et logarithme. Python : résolution numérique par balayage.	
19-20	Équations différentielles : notion, solutions de $y' = ay$ et $y' = ay + b$, condition initiale.	Spirale : dérivation, exponentielle. Python : méthode d'Euler.	PBL 1 (2h) : Décroissance radioactive (modélisation par équadiff).
21-22	Composition de fonctions : définition, dérivée $(v \circ u)' = u' \cdot (v' \circ u)$.	Spirale : dérivation (1ère). Python : calcul de dérivée de fonctions composées.	
23-24	Dérivée de $x \mapsto (u(x))^n$, $\cos(u(x))$, $\sin(u(x))$, $e^{u(x)}$, $\ln(u(x))$.	Spirale : composition (S21-22). Python : implémentation des formules.	Applications en physique (oscillations).
25-26	Primitives de $u' \cdot u^n$, $u' \cdot e^u$, $u' \cdot \cos u$, $u' \cdot \sin u$.	Spirale : primitives (S7-8) et composition (S23-24). Python : vérification par dérivation.	PBL 2 (2h) : Modélisation d'un circuit RLC (équation différentielle).

Contrôles continus : semaines 18, 22, 26 (avec exercice Python). **Formatives** : chaque semaine.

2.2.3 3^e trimestre (février – mai 2027) – 12 semaines

Sem.	Contenus nouveaux	Évaluation formative – points évalués (dont Python)	Remarques
27-28	Nombres complexes : exponentielle complexe $e^{i\theta} = \cos \theta + i \sin \theta$, forme exponentielle $z = re^{i\theta}$.	Spirale : formes trigonométrique (1ère). Python : conversion avec <code>cmath</code> .	Lien avec les signaux sinusoïdaux.
29-30	Formules d'addition et de duplication ; linéarisation de $\cos^2 \theta$ et $\sin^2 \theta$.	Spirale : trigonométrie (1ère). Python : calculs de primitives après linéarisation.	
31-32	Expression complexe des translations, rotations, homothéties.	Spirale : transformations du plan (1ère). Python : visualisation de transformations complexes.	PBL 3 (2h) : Étude d'un circuit électrique en régime sinusoïdal (impédances complexes).
33-34	Résolution d'équation du premier degré, de $z^2 = a$ ($a \in \mathbb{R}$).	Spirale : complexes (S27-28). Python : résolution numérique.	
35-36	Synthèse : intégration, exponentielle, logarithme, équations différentielles, complexes.	Bilan général. Python : projet complet de modélisation d'un phénomène physique.	Interdisciplinarité.
37-38	Révisions et évaluations finales.	Bilan : tous les thèmes + Python.	Préparation épreuves.

Contrôles continus : semaines 30, 34, 38 (avec exercice Python). **Formatives :** chaque semaine.

3 Évaluations

3.1 Évaluations formatives (hebdomadaires)

Chaque semaine, une courte activité (5–10 min) mêle mathématiques et Python. Les points évalués sont détaillés dans les tableaux ci-dessus. Ces évaluations ne sont pas notées, mais elles permettent une auto-évaluation et un suivi des compétences algorithmiques.

3.2 Évaluations sommatives (contrôles continus)

Au moins trois contrôles par trimestre, d'une durée de 1 h chacun. Chaque contrôle comporte :

- une partie sans calculatrice (calculs, raisonnement),
- une partie avec calculatrice / Python (exercice de programmation : écriture de fonction, complétion de code, simulation, représentation graphique, etc.).

Trimestre	Contrôle 1	Contrôle 2	Contrôle 3
1 ^{er} (sem. 5, 10, 14)	Intégration (aires, primitives, théorème fondamental) + Python (méthode des rectangles)	Exponentielle (limites, dérivées, fonctions) + Python (tracés, recherche de e)	Logarithme népérien (définition, propriétés, équations) + Python (calculs)
2 ^e (sem. 18, 22, 26)	Équations différentielles (linéaires du 1 ^{er} ordre) + Python (Euler)	Composition de fonctions (dérivées, primitives) + Python (formules)	Synthèse analyse + Python (projet modélisation)
3 ^e (sem. 30, 34, 38)	Nombres complexes (exponentielle, transformations) + Python (conversions, visualisation)	Complexes – résolutions d'équations, trigonométrie + Python (linéarisation)	Synthèse générale + Python (projet complet)

3.3 Problèmes basés sur l'apprentissage (PBL) – 2 h chacun

Trois PBL sont répartis sur l'année, en séances de 2 h (travail en groupes, compte rendu écrit et code Python rendu) :

1. **PBL 1 (semaine 20) : *Décroissance radioactive*.** Modéliser par une équation différentielle $y' = -\lambda y$, déterminer la solution, calculer la demi-vie, simuler avec Python (Euler et solution exacte).
2. **PBL 2 (semaine 26) : *Circuit RLC en régime libre*.** Établir l'équation différentielle du second ordre (sans la résoudre), étudier le régime pseudo-périodique à l'aide d'une simulation numérique (méthode d'Euler ou Runge-Kutta). Python : représentation des courbes.
3. **PBL 3 (semaine 32) : *Impédances complexes en régime sinusoïdal*.** Utiliser la notation exponentielle pour calculer l'impédance équivalente d'un circuit, déterminer le courant et la tension, tracer les diagrammes de Fresnel. Python : calculs et visualisation.

4 Devoirs maison (DM) – progression, Python et utilisation de l'IA

Afin de favoriser une acquisition à long terme, un devoir maison est distribué **toutes les deux semaines** (18 DM sur l'année). Chaque DM porte sur l'ensemble du programme traité depuis le 1er septembre, et comporte :

- des exercices de calcul et de raisonnement,
- un exercice de programmation Python,
- une mission spécifique d'utilisation de l'IA (consultation, débogage, confrontation de méthodes ou critique de réponse).

L'élève doit joindre à sa copie un **journal de bord** retraçant ses échanges avec l'IA et les décisions prises à la suite de ces échanges.

DM	Sem.	Thèmes évalués (mathématiques + Python)	Mission IA spécifique
1	3	Intégration : méthode des rectangles, aire sous la courbe. Python : somme de Riemann.	Demander à l'IA d'expliquer la convergence de la méthode des rectangles vers l'intégrale.
2	5	Intégration : propriétés, valeur moyenne. Python : calcul de la valeur moyenne.	Utiliser l'IA pour interpréter la valeur moyenne dans un contexte physique (puissance).
3	7	Théorème fondamental, primitives. Python : vérification de la dérivée de F .	Demander à l'IA de démontrer le théorème fondamental sur un exemple simple.
4	9	Exponentielle : e^x , dérivée, limites. Python : recherche de e par balayage.	Utiliser l'IA pour expliquer la croissance comparée et donner des exemples.
5	11	Dérivée de e^{kx} , étude de fonctions. Python : tracés.	Demander à l'IA de générer une fonction du type $e^{kx} + P(x)$ et d'en étudier les variations.
6	13	Logarithme népérien : définition, propriétés. Python : calculs de log.	Utiliser l'IA pour relier $\ln$ et $\log_{10}$.
7	16	Propriétés algébriques du logarithme. Résolution d'équations. Python : vérification.	Demander à l'IA de résoudre une équation mêlant exponentielle et logarithme.
8	18	Équations différentielles $y' = ay + b$. Python : méthode d'Euler.	Utiliser l'IA pour comparer la solution exacte et l'approximation d'Euler.
9	20	Composition de fonctions : dérivée $(v \circ u)'$. Python : calcul formel.	Demander à l'IA de démontrer la formule de dérivation d'une composée.
10	22	Dérivée de $\cos(u(x))$, $\sin(u(x))$, $e^{u(x)}$, $\ln(u(x))$. Python : vérification.	Utiliser l'IA pour identifier des compositions dans des fonctions complexes.

Suite à la page suivante

DM	Sem.	Thèmes évalués (mathématiques + Python)	Mission IA spécifique
11	24	Primitives de $u' \cdot u^n$, $u' \cdot e^u$, $u' \cdot \cos u$, $u' \cdot \sin u$. Python : vérification par dérivation.	Demander à l'IA de proposer une primitive pour une fonction du type $u'e^u$.
12	26	Nombres complexes : $e^{i\theta}$, forme exponentielle. Python : conversion.	Utiliser l'IA pour expliquer le lien entre forme exponentielle et rotation.
13	28	Formules d'addition, duplication, linéarisation. Python : calculs.	Demander à l'IA de linéariser $\cos^3 x$ et d'en donner une primitive.
14	30	Transformations complexes (translation, rotation, homothétie). Python : visualisation.	Utiliser l'IA pour interpréter géométriquement $z \mapsto az$ lorsque a est complexe de module 1.
15	32	Résolution de $z^2 = a$ pour a réel. Python : résolution numérique.	Demander à l'IA de résoudre une équation du type $z^2 = -4$ et d'expliquer les solutions.
16	34	Synthèse intégration et exponentielle/log. Python : projet de modélisation.	Utiliser l'IA pour critiquer la méthode d'intégration utilisée dans un problème physique.
17	36	Synthèse équations différentielles et complexes. Python : projet complet.	Demander à l'IA un retour sur la structure du code et proposer des extensions.
18	38	Bilan général. Python : projet complet.	Demander à l'IA de suggérer des applications en physique-chimie.

Remarques :

- Les DM sont à rendre une semaine après la distribution.
- La mission IA doit être clairement identifiée dans le journal de bord (copie d'écran ou retranscription des questions/réponses, suivie de l'analyse personnelle de l'élève).
- La correction est effectuée en classe, avec une attention particulière aux erreurs fréquentes, aux méthodes de résolution et aux bonnes pratiques d'utilisation de l'IA.

5 Conseils pour la mise en œuvre

- **Rituels** : consacrer 5 à 10 minutes par séance à des exercices de calcul (mental, littéral) et à de courts extraits de code Python (lecture, complétion, débogage).
- **Travail en îlots** : favoriser les échanges entre élèves lors des phases de recherche et des PBL.
- **Numérique** : utiliser Python régulièrement (au moins une séance sur deux). Insister sur la programmation modulaire (fonctions) et la documentation du code.
- **IA éthique** : former les élèves à formuler des requêtes précises et à évaluer de manière critique les réponses de l'IA (vérification des calculs, cohérence, limites). Insister sur le fait que l'IA est un *outil d'aide*, pas un substitut au raisonnement personnel.

- **Différenciation** : proposer des exercices de difficulté progressive et des approfondissements pour les élèves plus à l’aise (notamment en programmation et en interaction avec l’IA).
- **Trace écrite** : veiller à ce que chaque élève dispose d’un cahier de cours structuré (définitions, propriétés, exemples) et d’un cahier de programmation regroupant les fonctions et scripts clés.

Conclusion

Cette programmation respecte les 31 semaines de cours disponibles et couvre l’intégralité du programme de mathématiques de la spécialité PCM en terminale STI2D, avec une intégration systématique de Python et de l’utilisation critique de l’IA dans les apprentissages et les évaluations. La progressivité spiralaire, appliquée aussi aux compétences algorithmiques et numériques, garantit que les notions mathématiques, la programmation et le jugement critique sont solidement maîtrisés en fin d’année. Les évaluations variées (formatives, sommatives, PBL, DM) permettent de suivre la progression de chaque élève et de développer ses compétences de modélisation, de raisonnement, de communication, de codage et de collaboration avec les outils d’intelligence artificielle.